T3. Transformaciones Geométricas, 2D-3D

Transformaciones

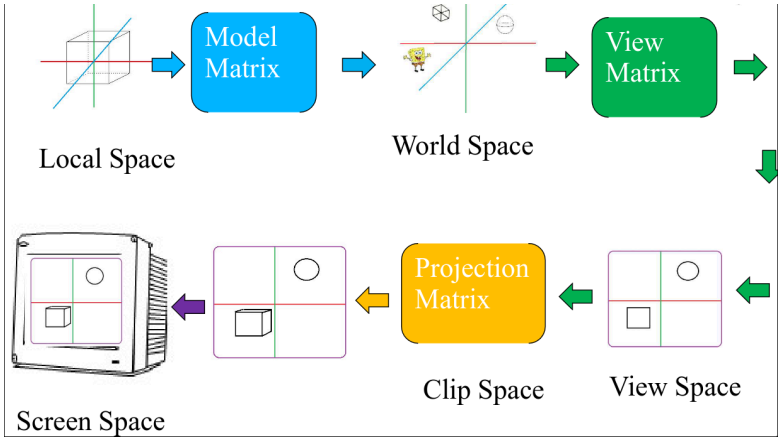
En la **pipeline gráfica** se describen varios espacios de trabajo:

- **Local Space** es el sistema de **coordenadas locales** de los objetos con relación a su **origen local**.

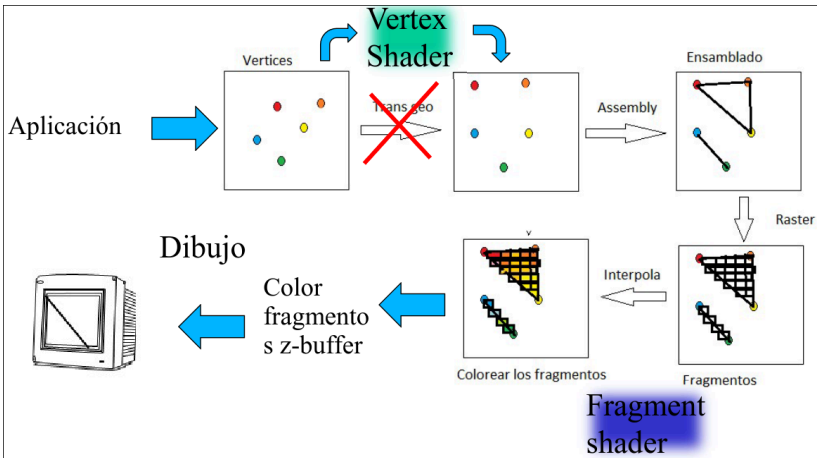
- **World Space** es el sistema de **coordenadas de escena** o de mundo de los objetos con relación al origen global.

La **matriz del modelo** (Model Matrix) **transforma** las coordenadas **locales** en coordenadas **de escena** comunes a todos los objetos, **trasladando**, **rotando** y **escalando** los objetos hasta colocarlos en su posición final.

Las **transformaciones geométricas** se usan para trasladar, escalar y rotar los objetos y la cámara de la escena.



En la pipeline fija (Opengl 1.2) el paso de coordenadas locales a coordenadas de la escena se corresponde con el paso **transformación geométrica**. En el retained mode (Opengl 3.3) el paso de coordenadas locales a coordenadas de escena se realiza mediante el **Vertex Shader**.

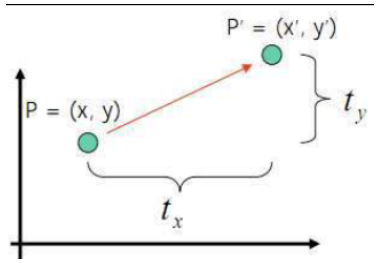


- <u>Traslación</u>

Llamamos **traslación** a mover un objeto (todos sus vértices) de un lugar a otro. Si queremos trasladar un punto $P\ (x\ ,\ y)$ a una nueva posición $P'\ (x'\ ,\ y')$, sumamos a sus coordenadas $(x\ ,\ y)$ las cantidades que lo queremos trasladar, T_x, T_y (vector de desplazamiento, donde: $P' = P + T$.

- $x' = x + T_x$
- $y' = y + T_y$

$$\begin{bmatrix} x' \\ y' \end{bmatrix} = \begin{bmatrix} x \\ y \end{bmatrix} + \begin{bmatrix} T_x \\ T_y \end{bmatrix} = \begin{bmatrix} x + T_x \\ y + T_y \end{bmatrix}$$



Si queremos trasladar un objeto trasladaremos cada vértice del objeto. En Opengl se usa $glTranslatef(Tx,\ Ty,\ Tz)$.

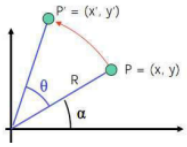
- <u>Rotación respecto al Origen</u>

Llamemos **rotación** respecto al origen a desplazar un **ángulo θ** las coordenadas de un punto respecto al origen. Rotamos una línea u objeto rotando cada uno de sus vértices. Para rotar un punto $P\ (x\ ,\ y)$ usamos relaciones trigonométricas:

- $x = r \cos(\alpha)$;
- $y = r \operatorname{sen}(\alpha)$;
- $x' = r \cos(\alpha + \theta)$;
- $y' = r \operatorname{sen}(\alpha + \theta)$;

$x' = x \,.\, \cos(\alpha) - y \,.\, \operatorname{sen}(\theta)$;
$y' = x \,.\, \operatorname{sen}(\alpha) + y \,.\, \cos(\theta)$;

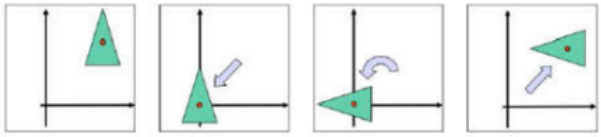
$$\begin{bmatrix} x' \\ y' \end{bmatrix} = \begin{bmatrix} \cos\theta & -sen\theta \\ sen\theta & \cos\theta \end{bmatrix} \cdot \begin{bmatrix} x \\ y \end{bmatrix}$$



En Opengl $glRotatef(\theta,\ x,\ y, z)$ donde θ está en grados y el giro se realiza alrededor del eje que pasa por $(0,\ 0, 0)$ y $(x,\ y, z)$ en sentido contrario a las agujas del reloj.

- <u>Rotación General</u>

Para desplazar un ángulo θ un objeto (todos sus vertices) respecto un **punto cualquiera** que no es el origen $(x_c,\ y_c)$: se debe realizar una traslación al origen, rotación respecto al origen y traslación a $(x_c,\ y_c)$.



$$P' = T(x,\ y)\ *\ R(\alpha)\ *\ T(-\ x,\ -\ y)\ *\ P(x, y)$$

- <u>Ángulos de Euler</u>

Dados dos sistemas de coordenadas $x,\ y,\ z$ y $X,\ Y,\ Z$ con un **mismo origen**, es posible determinar la posición de uno en términos del otro usando **tres ángulos**

α, β, γ (yaw, pitch and roll). Esto sirve para representar los ejes de un objeto respecto a un **sistema de referencia general**.

Los giros son **acumulativos** y los **ángulos** se encuentran **ya girados**.

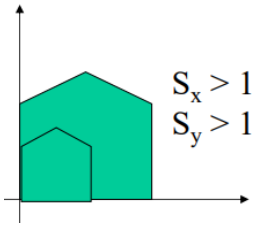
- <u>Escalado respecto al origen</u>

Llamamos escalado a **cambiar** el **tamaño de un objeto** (todos sus vértices). No es una transformación de cuerpo rígido ya que la geometría varía. Para escalar **multiplicamos** por los **coeficientes** de escalado, que serán la razón entre las coordenadas nuevas y viejas:

- $S_x = anchura\ nueva\ /\ anchura\ vieja.$
- $S_y = altura\ nueva\ /\ altura\ vieja.$
- Si $S_x = S_y$ se realiza un **escalado uniforme**, si $S_x \neq S_y$ se realiza un **escalado no uniforme.**

$$0 < S_x < 1$$
$$0 < S_y < 1$$

Para escalar un punto $P\,(x\,,\,y)$ multiplicamos sus coordenadas por los coeficientes de escalado:



$x' = S_x . x$
$y' = S_y\ y;$

$$\begin{bmatrix} x' \\ y' \end{bmatrix} = \begin{bmatrix} S_x & 0 \\ 0 & S_y \end{bmatrix} \cdot \begin{bmatrix} x \\ y \end{bmatrix}$$

$$\begin{bmatrix} x' \\ y' \end{bmatrix} = \begin{bmatrix} S_x \cdot x \\ S_y \cdot y \end{bmatrix}$$

En openGl $glScale(Sx,\ Sy,\ Sz)$.

- <u>Escalado General</u>

Para escalar un objeto (todos sus vértices) cuando el origen de coordenadas no está en el interior del objeto, se produce un desplazamiento. El escalado de un objeto fuera del origen implica variar su posición. Para evitarlo debemos sustituir el simple escalado por una **composición de transformaciones**: traslación al origen, escalado respecto al origen y traslación al punto inicial.

$$\begin{cases} x' = x_c + S_x \cdot (x - x_c) \\ y' = y_c + S_y \cdot (y - y_c) \end{cases}$$

$$P' = T(x,\ y)\ *\ S(S_x,\ S_y)\ *\ T(-\ x,\ -\ y)\ *\ P(x, y)$$

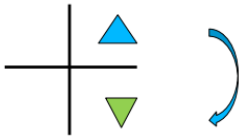
- <u>Otras transformaciones</u>

Denominaremos otras transformaciones a aquellas que hacen uso de la matriz (2D).

Reflexión

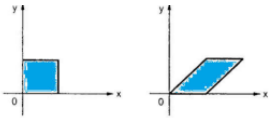
$$\begin{bmatrix} r_{11} & r_{12} \\ r_{21} & r_{22} \end{bmatrix}$$

$$\begin{bmatrix} 1 & 0 \\ 0 & -1 \end{bmatrix}$$



Shear

$$\begin{bmatrix} 1 & a \\ 0 & 1 \end{bmatrix}$$



$$\begin{bmatrix} 1 & a \\ b & -1 \end{bmatrix}$$

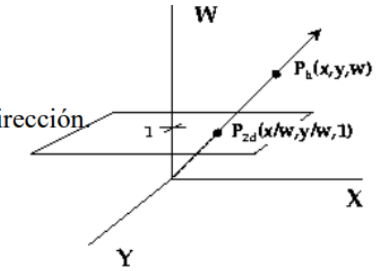
Se hace share en las dos direcciones.

Coordenadas Homogéneas

Para poder realizar todas las **transformaciones geométricas** (translaciones, rotaciones y escalados) exclusivamente mediante **multiplicación de matrices** hay que cambiar la forma en la que se realiza la traslación. Para esto utilizaremos las **coordenadas homogéneas**, que añaden un nuevo elemento w que será la última columna de las matrices de transformación, la cual se usará para colocar los factores de translación.

Punto P(x ,y ,z, w)

- Vector P (x, y, z, 0) recordar es una dirección
- Coordenadas homogéneas w =1
- En general w = 1
- No existe (0, 0, 0), (0, 0, 0, 1).
- Punto (x, y, z, 1), vector (x, y, z, 0)



- <u>Transformaciones en 2D</u>

$$T(x,y) = T(x,y,1)$$

$$\begin{bmatrix} x' \\ y' \end{bmatrix} = \begin{bmatrix} x \\ y \end{bmatrix} + \begin{bmatrix} T_x \\ T_y \end{bmatrix} = \begin{bmatrix} x + T_x \\ y + T_y \end{bmatrix} \quad \Rightarrow \quad \begin{pmatrix} 1 & 0 & T_x \\ 0 & 1 & T_y \\ 0 & 0 & 1 \end{pmatrix}$$

$$\begin{bmatrix} x' \\ y' \end{bmatrix} = \begin{bmatrix} \cos\theta & -\sin\theta \\ \sin\theta & \cos\theta \end{bmatrix} \cdot \begin{bmatrix} x \\ y \end{bmatrix} \quad \Rightarrow \quad \begin{pmatrix} \cos\alpha & -\sin\alpha & 0 \\ \sin\alpha & \cos\alpha & 0 \\ 0 & 0 & 1 \end{pmatrix}$$

$$\begin{bmatrix} x' \\ y' \end{bmatrix} = \begin{bmatrix} S_x & 0 \\ 0 & S_y \end{bmatrix} \cdot \begin{bmatrix} x \\ y \end{bmatrix} \quad \Rightarrow \quad \begin{pmatrix} S_x & 0 & 0 \\ 0 & S_y & 0 \\ 0 & 0 & 1 \end{pmatrix}$$

- <u>Transformaciones en 3D</u>

Traslación

$$T(d_x, d_y, d_z) = \begin{bmatrix} 1 & 0 & 0 & d_x \\ 0 & 1 & 0 & d_y \\ 0 & 0 & 1 & d_z \\ 0 & 0 & 0 & 1 \end{bmatrix}$$

Escalado

$$S(s_x, s_y, s_z) = \begin{bmatrix} s_x & 0 & 0 & 0 \\ 0 & s_y & 0 & 0 \\ 0 & 0 & s_z & 0 \\ 0 & 0 & 0 & 1 \end{bmatrix}$$

Rotación alrededor del eje Z

$$R_z(\theta) = \begin{bmatrix} \cos\theta & -\sin\theta & 0 & 0 \\ \sin\theta & \cos\theta & 0 & 0 \\ 0 & 0 & 1 & 0 \\ 0 & 0 & 0 & 1 \end{bmatrix}$$

Rotación alrededor del eje X

$$R_x(\theta) = \begin{bmatrix} 1 & 0 & 0 & 0 \\ 0 & \cos\theta & -\sin\theta & 0 \\ 0 & \sin\theta & \cos\theta & 0 \\ 0 & 0 & 0 & 1 \end{bmatrix}$$

Rotación alrededor del eje Y

$$R_y(\theta) = \begin{bmatrix} \cos\theta & 0 & \sin\theta & 0 \\ 0 & 1 & 0 & 0 \\ -\sin\theta & 0 & \cos\theta & 0 \\ 0 & 0 & 0 & 1 \end{bmatrix}$$

- <u>Orden de aplicación</u>

- Caso A: Rotar y luego Escalar
  Primero giramos el punto y luego lo escalamos
  - **Rotación**: el punto $(1,\ 0)$ pasa a ser $(0,\ 1)$.
  - **Escalado**: aplicamos escalado al resultado $(0,\ 1)$. Como el escalado es en X, y nuestra X ahora es 0, el punto no cambia de posición.

- **Rotación (90°):** $R = \begin{pmatrix} 0 & -1 \\ 1 & 0 \end{pmatrix}$
- **Escalado (x2 en X):** $S = \begin{pmatrix} 2 & 0 \\ 0 & 1 \end{pmatrix}$
- **Punto Original:** $P = \begin{pmatrix} 1 \\ 0 \end{pmatrix}$

1. Cálculo de la Matriz Resultante (M_{SR}):

$$M_{SR} = \begin{pmatrix} 2 & 0 \\ 0 & 1 \end{pmatrix} \cdot \begin{pmatrix} 0 & -1 \\ 1 & 0 \end{pmatrix} = \begin{pmatrix} (2 \cdot 0 + 0 \cdot 1) & (2 \cdot -1 + 0 \cdot 0) \\ (0 \cdot 0 + 1 \cdot 1) & (0 \cdot -1 + 1 \cdot 0) \end{pmatrix} = \begin{pmatrix} 0 & -2 \\ 1 & 0 \end{pmatrix}$$

2. Multiplicación por el punto P:

$$P' = M_{SR} \cdot P = \begin{pmatrix} 0 & -2 \\ 1 & 0 \end{pmatrix} \cdot \begin{pmatrix} 1 \\ 0 \end{pmatrix} = \begin{pmatrix} (0 \cdot 1 + -2 \cdot 0) \\ (1 \cdot 1 + 0 \cdot 0) \end{pmatrix} = \begin{pmatrix} 0 \\ 1 \end{pmatrix}$$

- **Caso B: Escalar y luego Rotar**
  Primero estiramos el punto y luego lo giramos.
  - **Escalado**: el punto $(1,\ 0)$ pasa a ser $(2,\ 0)$.
  - **Rotación**: aplicamos R al resultado $(2,\ 0)$. El punto gira 90° sobre el origen.

1. Matriz resultante:

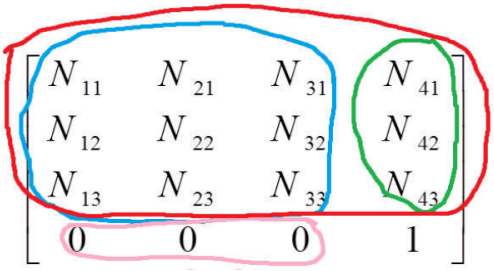
$$M_{RS} = \begin{pmatrix} 0 & -1 \\ 1 & 0 \end{pmatrix} \begin{pmatrix} 2 & 0 \\ 0 & 1 \end{pmatrix} = \begin{pmatrix} 0 & -1 \\ 2 & 0 \end{pmatrix}$$

2. Aplicación al punto:

$$P'' = \begin{pmatrix} 0 & -1 \\ 2 & 0 \end{pmatrix} \begin{pmatrix} 1 \\ 0 \end{pmatrix} = \begin{pmatrix} 0 \\ 2 \end{pmatrix}$$

- <u>Zonas de una matriz</u>

La zona azul serían transformaciones lineales, la verde traslaciones y la rosa ceros. Con OpenGL se usa $glMatrixMode(GL_MODELVIEW)$.



Matrices de transformación con GLM

En OpenGL moderno, las transformaciones de los objetos se realizan utilizando matrices. Para ello se emplea la **librería GLM** (OpenGL Mathematics), que permite trabajar fácilmente con vectores y matrices.

Una **matriz de transformación** típica es de tipo $glm::mat4$ (matriz $4x4$), que permite representar traslaciones, rotaciones y escalados en el espacio 3D. El proceso comienza siempre creando una **matriz identidad** ($glm::mat4(1.0f)$), que actúa como punto de partida. A partir de ella, se aplican **transformaciones** como $glm::translate(matriz\ transform,\ glm::vec3(datos))$ para **mover** el objeto, $glm::rotate(matriz\ transform,\ angulo,\ glm::vec3(datos))$ para **girarlo** y $glm::scale(matriz\ transform,\ glm::vec3(datos))$ para cambiar su **tamaño**.

- <u>Importancia del orden de las transformaciones</u>

El **orden** en el que se aplican las transformaciones es fundamental, ya que modifica completamente el resultado final. Aunque en el código las transformaciones se escriben en un orden determinado, en realidad se **aplican**

en orden inverso debido a la multiplicación de matrices. Por ejemplo, si primero se traslada y luego se rota, el objeto girará sobre su propio eje en su nueva posición. Sin embargo, si primero se rota y luego se traslada, el objeto girará alrededor del origen de coordenadas.

- <u>Uso de radianes en rotaciones</u>

Las funciones de rotación en GLM trabajan en **radianes**, no en grados. Por ello, cuando se desea usar ángulos en grados, es necesario convertirlos utilizando la función $glm::radians$.

- <u>Comunicación entre CPU y GPU: uniforms</u>

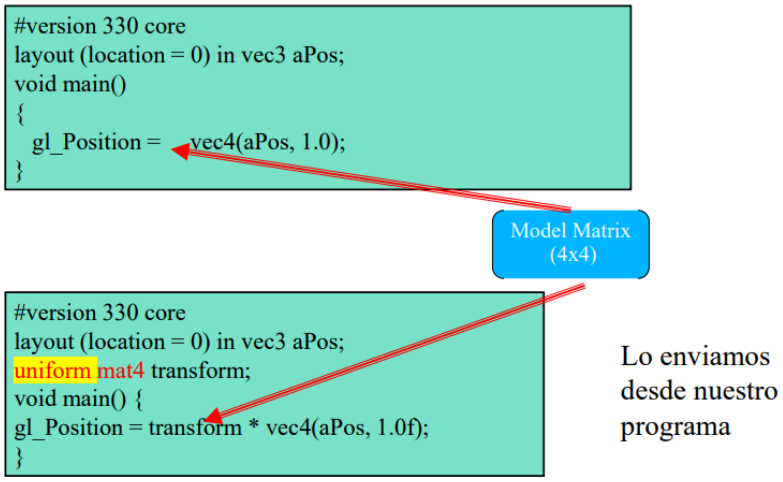
Para aplicar transformaciones en el pipeline gráfico, es necesario **enviar** las matrices desde la CPU (el programa principal) a la **GPU** (donde se ejecutan los shaders). Esto se realiza mediante variables llamadas **uniforms**.

Un **uniform** es una variable **global** dentro del **shader** que mantiene el **mismo valor para todos los vértices** durante una llamada de dibujo. Se utilizan habitualmente para matrices de transformación, colores o variables de tiempo. A diferencia de los atributos, que cambian para cada vértice (attribute), los uniforms permanecen constantes para todo el objeto.

El uso de uniforms sigue tres pasos fundamentales. Primero, se **declara la variable** en el shader, normalmente en el Vertex Shader, fuera de la función principal. Después, en el programa principal, se obtiene la **localización** de esa variable mediante la función:

$$int\ loc = glGetUniformLocation\ (programaID,\ "nombre\ uniform\ mat4"),$$

que devuelve una referencia a su posición en memoria. Finalmente, se envía el **valor** a la GPU utilizando funciones como $glUniformMatrix4fv\ (localización,\ \ldots)$, que permite transferir matrices. Este proceso debe realizarse antes de dibujar el objeto.

- <u>Aplicación de la matriz en el Vertex Shader</u>

Una vez que la matriz ha sido enviada al shader, se utiliza en el Vertex Shader para **transformar los vértices**. Esto se hace multiplicando la matriz por la posición del vértice, utilizando la variable $gl_Position$. De esta forma, cada vértice es transformado según las operaciones definidas (traslación, rotación y escala), y el resultado es la **posición final** que se mostrará en pantalla.